Image Processing

Lecture 2: Image Processing Basics

By

Dr. Rasha Mahmoud Kamel

Digital Image Representation – Image Layout

- **A digital image** can be considered as ***a discrete representation of data*** possessing both spatial (layout) and intensity (color) information.
- **A digital image** can be ***represented as a two-dimensional (2D) matrix*** of real numbers.
- Suppose that I represents a 2D discrete digital image of size $M \times N$ and $I(m, n)$ represents the intensity or color at a series of pixel positions $(m = 1,2, \dots, M\,; n = 1,2, \dots, N)$ in 2D Cartesian coordinates.
- The indices m and n respectively designate the rows and columns of the image.
- The individual ***picture elements or pixels*** of the image are thus referred to by their 2D (m, n) index.

Digital Image Representation – Image Layout

Digital Image Representation – Image Layout

- Following the **MATLAB** convention, ***monochrome images are essentially 2D matrices*** (or arrays) where $I(m,n)$ denotes the intensity or gray level of the image at the pixel located at the m^{th} row and n^{th} column starting from ***a top-left image origin***.
- **MATLAB** and its Image Processing Toolbox (IPT) use ***1-based array notation***. A monochrome image matrix looks like this:

$$I(m,n) = \begin{bmatrix} I(1,1) & I(1,2) & \cdots & I(1,N) \\ I(2,1) & I(2,2) & \cdots & I(2,N) \\ \vdots & \vdots & \cdots & \vdots \\ I(M,1) & I(M,2) & \cdots & I(M,N) \end{bmatrix}$$

Digital Image Representation – Image Color

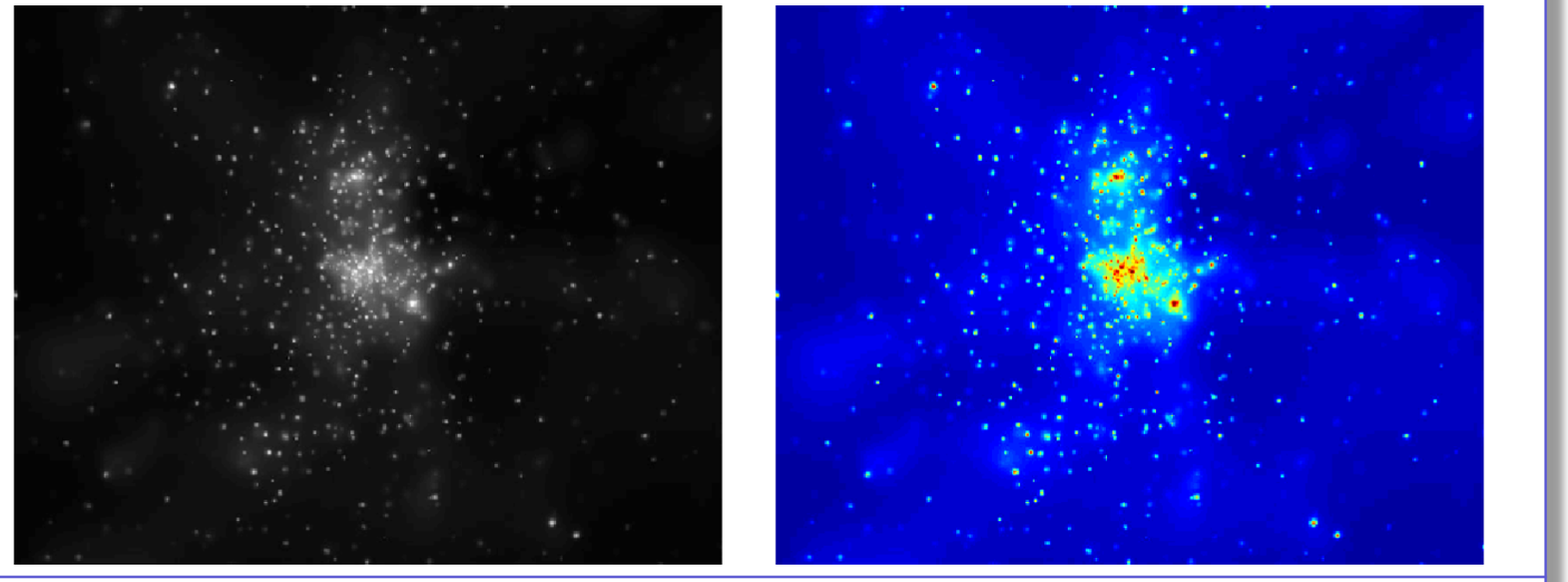
- **An image** ***contains one or more color channels*** that define the intensity or color at a particular pixel location $I(m, n)$.
- In the simplest case, ***each pixel location only contains a single numerical value*** representing the signal level at that point in the image.
- The conversion from this set of numbers to an actual (displayed) image is achieved through a ***color map***.
- **A color map** ***assigns a specific shade of color to each numerical level in the image*** to give a visual representation of the data.
- The most common ***color map is the greyscale***, ***which is particularly well suited to intensity images*** (images that express the intensity of the signal as a single value at each pixel).

Digital Image Representation – Image Color

- **The greyscale color map** ***assigns all shades of grey from black (zero) to white (maximum)*** according to the signal level.
- In certain instances, it can be better to display ***intensity images using a false-color map***.
- One of the main motives behind the use of false-color display rests on the fact that the ***human visual system is only sensitive to approximately 40 shades of grey*** in the range from black to white, whereas our sensitivity to color is much finer.
- False color can also serve to ***accentuate or delineate certain features or structures***, making them easier to identify for the human observer.
- This approach is often taken in medical and astronomical images.

Digital Image Representation – Image Color

- This figure shows an astronomical intensity image displayed using both ***greyscale and a particular false-color map***. In this example the ***jet color map*** (as defined in MATLAB) has been used to ***highlight the structure and finer detail of the image*** to the human viewer using a linear color scale ranging from dark blue (low intensity values) to dark red (high intensity values).



Example of grayscale (left) and false color (right) image display

Digital Image Representation – Image Color

- In addition to greyscale images where we have a single numerical value at each pixel location, we also have ***truecolor images*** where the full spectrum of ***colors can be represented as a triplet vector***, typically the (R,G,B) components at each pixel location.
- Here, the color is represented as a linear combination of the basis colors or values and the image may be considered as consisting of three 2-D planes.
- ***Other representations of color*** are also possible and used quite widely, ***such as the (H,S,V)*** (hue, saturation and value (or intensity)). In this representation, the intensity V of the color is decoupled from the chromatic information, which is contained within the H and S components.

Resolution and Quantization

- The ***size of the 2D pixel grid together with the data size stored for each individual image pixel*** determines the **spatial resolution and color quantization of the image**.
- The representational power (or size) of an image is defined by its resolution.
- ***The resolution of an image source*** (e.g. a camera) can be specified in terms of three quantities:

  (1) Spatial resolution: ***The number of pixels*** used to cover the visual space captured by the image is defined by row (M) by the column (N) dimensions of the image. This relates to the sampling of the image signal and is sometimes referred to as the pixel or digital resolution of the image. It is commonly quoted as $M \times N$ (e.g. 640×480, 800×600, 1024×768, etc.) .

Resolution and Quantization

(2) Temporal resolution: For a continuous capture system such as video, ***this is the number of images captured in a given time period***.

It is commonly quoted in ***frames per second*** (fps), where each individual image is referred to as a video frame (e.g. commonly broadcast TV operates at 25 fps; 25–30 fps is suitable for most visual surveillance; higher frame-rate cameras are available for specialist science/engineering capture).

Resolution and Quantization

(3) Bit resolution: This defines ***the number of possible intensity/color values that a pixel may have*** and relates to the quantization of the image information.

For instance a ***binary image*** has just two colors (black or white), a ***greyscale image*** commonly has 256 different grey levels ranging from black to white whilst for a ***color image*** it depends on the color range in use.

The bit resolution is commonly quoted as the number of binary bits required for storage at a given quantization level, e.g. binary is 1 bit, greyscale is 8 bit and color (most commonly) is 24 bit.

The range of values a pixel may take is often referred to as the ***dynamic range of an image***.

Image Formats

- From a mathematical viewpoint, any meaningful ***2D array of numbers can be considered as an image***.
- In the real world, we need to effectively display images, store them, transmit them over networks, etc. This has led to the development of standard digital image formats.
- **Image file format** is a ***standard way to organize and store image data***. It defines how the data is arranged and the type of compression—if any—that is used.
- **The image formats** comprise a ***file header and the actual numeric pixel values*** themselves.
- The image **file header** ***stores information about the image*** such as image height and width, number of bits per pixel, some signature bytes indicating the file type, etc.

Image Formats

- There are a large number of recognized image formats now existing. Common image formats and their associated properties are listed in the following table.

Acronym	Name	Properties
GIF	Graphics interchange format	Limited to only 256 colors (8 bit); ***lossless compression***
JPEG	Joint Photographic Experts Group	In most common use today; ***lossy compression***
BMP	Bit map picture	Basic image format; ***lossless compression***
PNG	Portable network graphics	New ***lossless compression*** format
TIF/TIFF	Tagged image (file) format	Highly flexible, detailed and adaptable format; ***both lossy and lossless compression***

Image Formats

- Different image formats are generally suitable for different applications.
- **GIF** is a very basic image storage format that supports grayscale and color images. It is ***limited to only 256 grey levels or colors*** (8 bits per pixel) . It uses an indexed representation for color images with a color map of a maximum of 256 colors.
- **JPEG format** supports grayscale and color images. It supports ***up to a 24-bit RGB color image*** with up to 16 million colors, and up to 36 bits for medical/scientific imaging applications. It is the most popular file format for photographic quality image representation (e.g. digital cameras). It is capable of achieving ***high degrees of compression*** with minimal perceptual loss of quality.

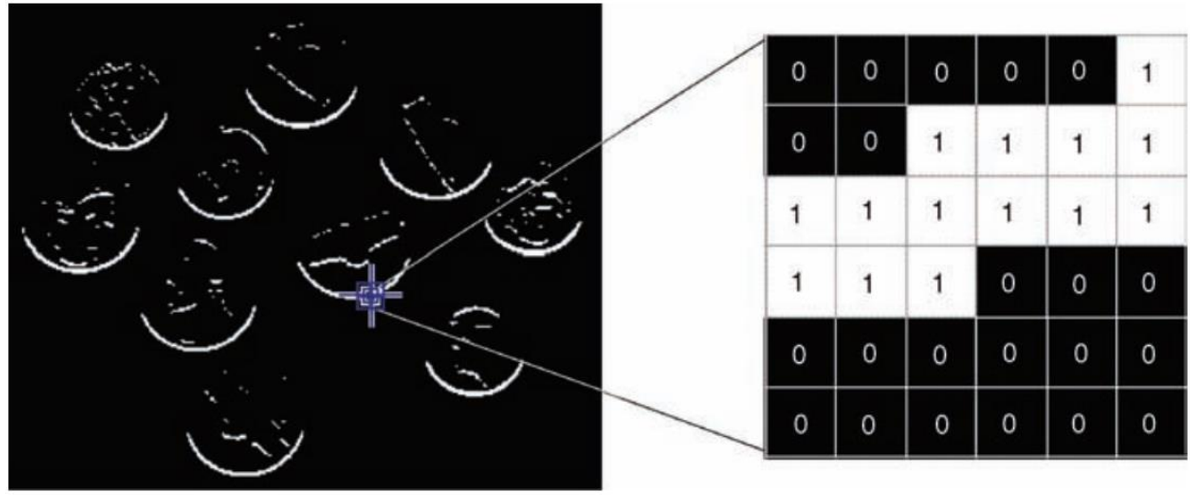
Image Formats

- **PNG format** is an increasingly popular file format that designed to replace GIF. It supports both ***indexed images*** (with up to 256 colors), ***grayscale images*** (with up to 16 bits per pixel), and ***truecolor images*** (with up to 3×16 bits per pixel).
- **BMP format**, originating in the development of the ***Microsoft Windows operating system***, supports grayscale, indexed, and truecolor images. BMP and PNG format are designed as a more powerful replacement for GIF.
- **TIFF** is a more sophisticated format with many options and capabilities, including the ability to represent 24-bit truecolor image.

Image Data Types – Binary Images

- **Binary images** are 2D arrays that assign one numerical value from the set {0,1} to each pixel in the image. These are sometimes referred to as logical images. Black corresponds to zero (an 'off' or 'background' pixel) and white corresponds to one (an 'on' or 'foreground' pixel).
- Binary images are typically represented in **MATLAB** using a logical array of 0's and 1's.

Image: binary
Pixel Data Type: integer (0 or 1)
Image Format: PNG



Binary image and the pixel values in a 6 × 6 neighborhood

Image Data Types – Greyscale Images

- **Intensity or greyscale images** are 2D arrays that assign one numerical value to each pixel which is representative of the intensity at this point.
- The pixel value range is bounded by the bit resolution of the image and such images are stored as N-bit integer images.
- Each pixel can be assigned whole integers in the range of $[0 \ldots 2^k - 1]$.
- A typical grayscale image uses $k = 8$ bits (1 byte) per pixel and intensity values in the range of [0 ... 255], where the value 0 corresponds to “black”, 255 means “white”, and intermediate values indicate varying shades of gray.

Image Data Types – Greyscale Images

- Intensity images can be represented in **MATLAB** using different data types (uint8, uint16, double). For greyscale images with elements of integer classes uint8 and uint16, each pixel has a value in the [0, 255] and the [0, 65,535] range, respectively. Greyscale images of class double have pixel values in the [0.0, 1.0] range.

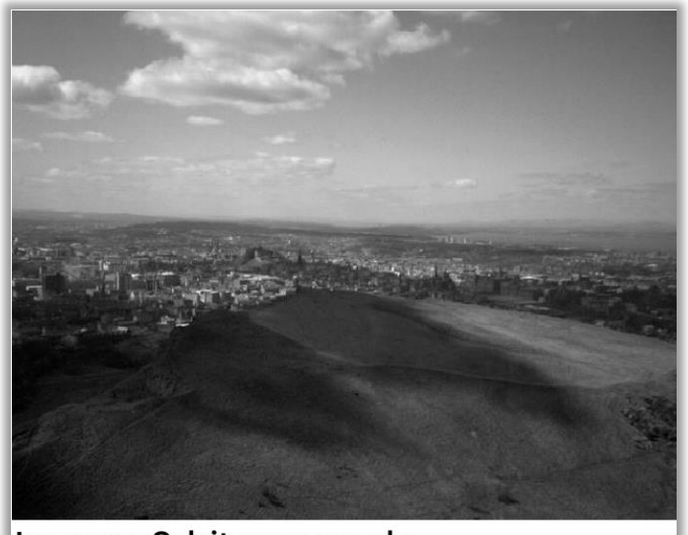
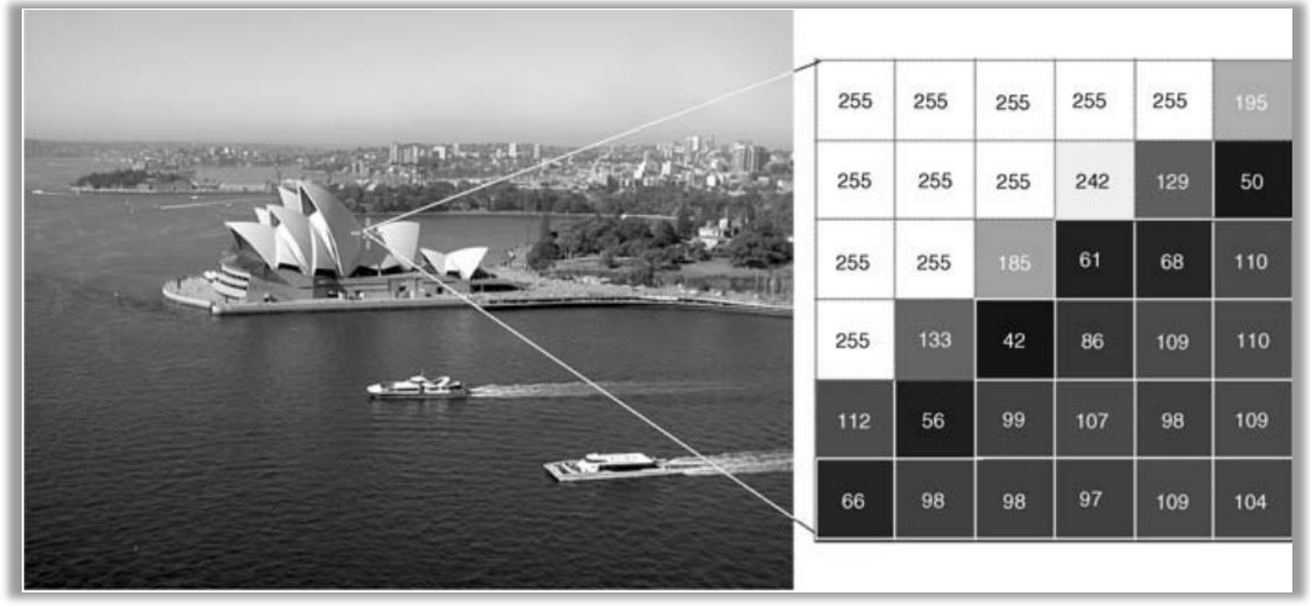


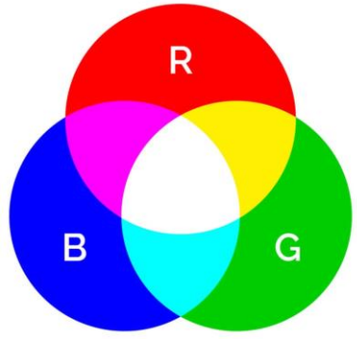
Image: 8-bit grayscale
Pixel Data Type: integer (0→255)
Image Format: GIF



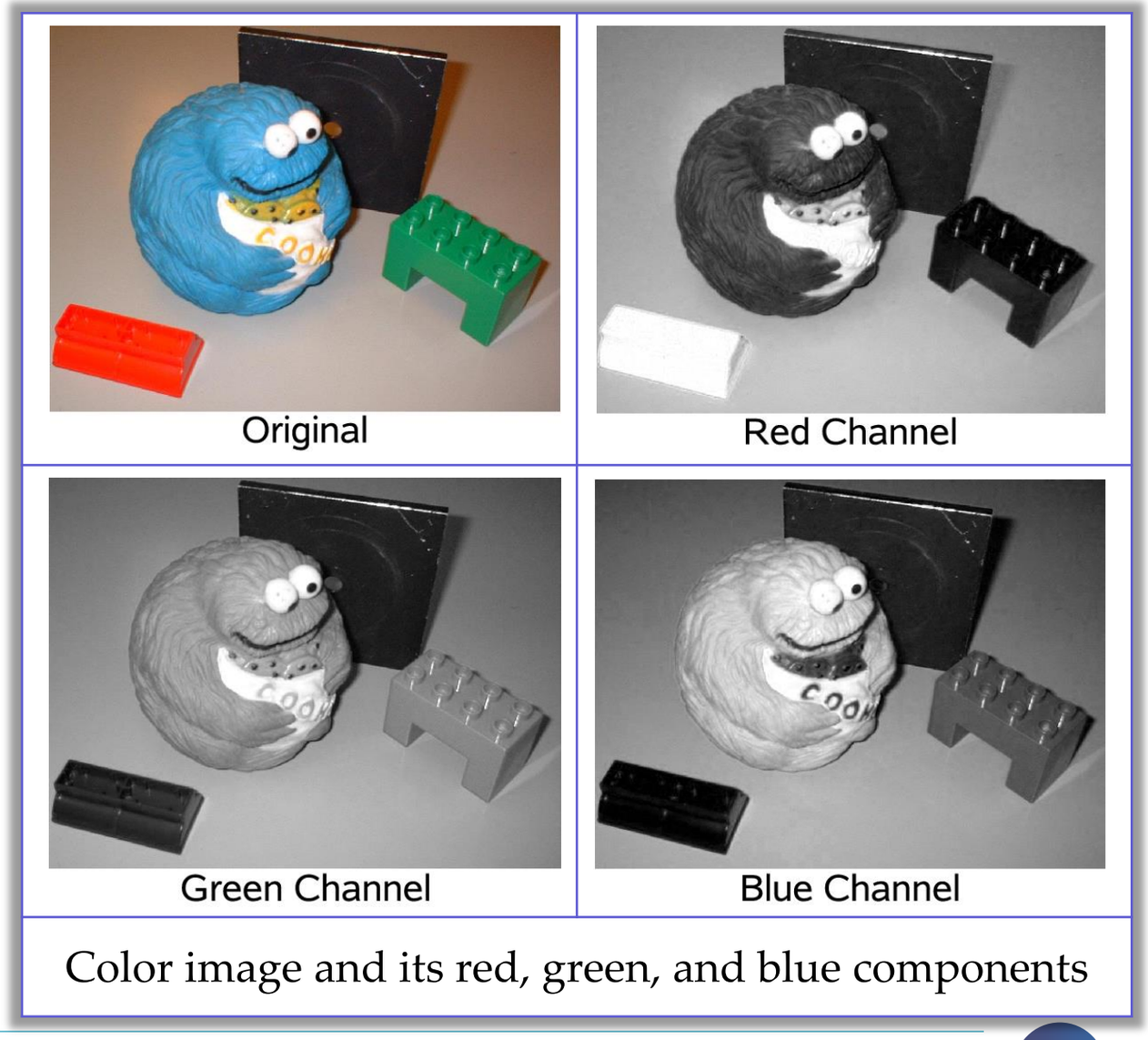
A grayscale image and the pixel values in a 6 × 6 neighborhood

Image Data Types – Color Images

- Most **color images** are based on the primary colors ***red, green, and blue (RGB).***



- Color images can be represented using ***three 2D arrays of same size***, one for each color channel: red (R), green (G), and blue (B).



Color image and its red, green, and blue components

Image Data Types – Color Images

- **Each array** element contains ***an 8-bit value***, indicating the amount of red, green, or blue at that point.
- ***Each pixel requires*** $3 \times 8 = 24$ bits to encode all three components, and the range of each individual color component is [0 ... 255].
- The combination of the three 8-bit values into a ***24-bit number allows*** 2^{24} (16,777,216) color combinations.

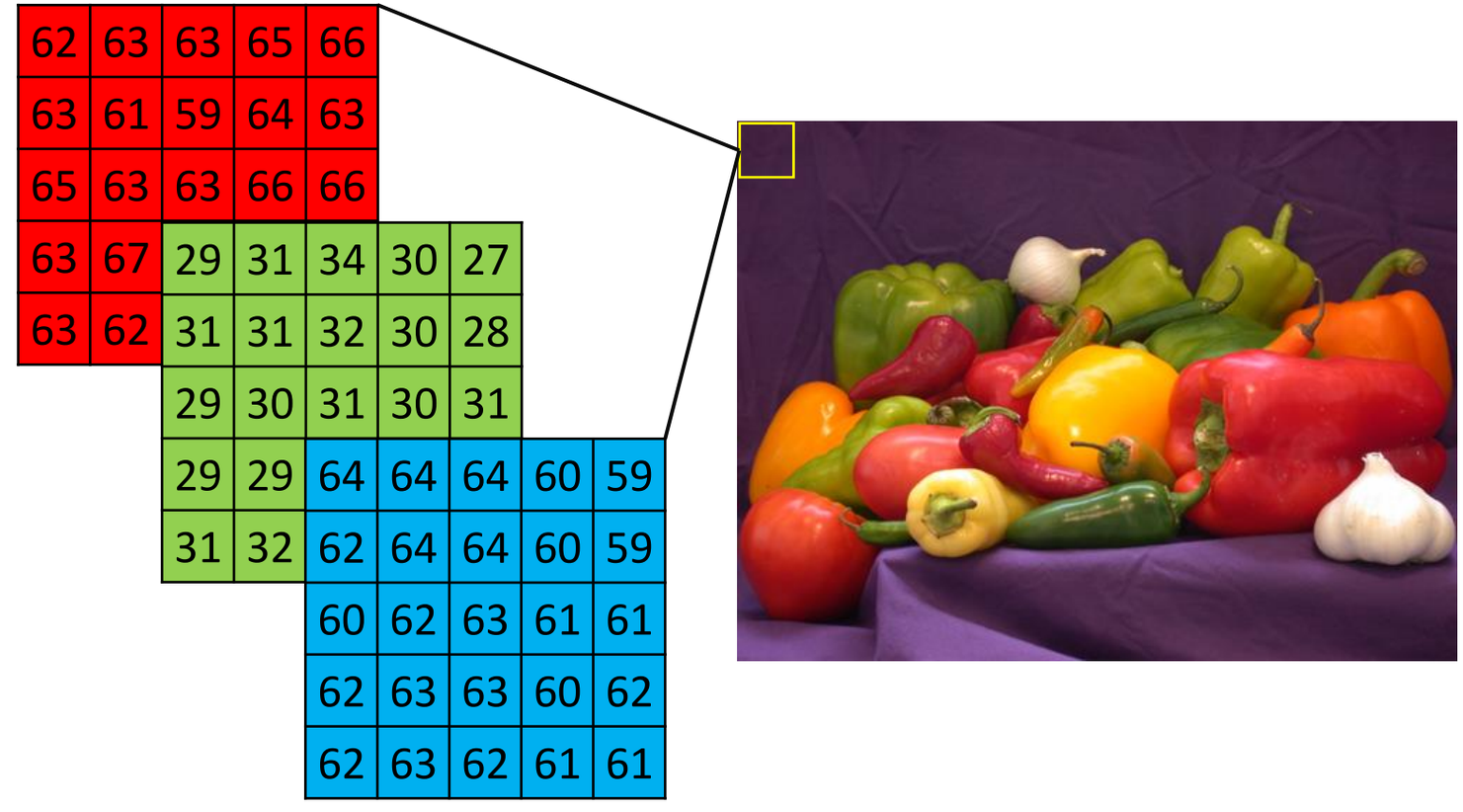


Image Data Types – Color Images

- RGB color images ***are represented in MATLAB as 3D arrays*** of dimension $(M \times N \times 3)$, where M denotes the number of image rows and N the number of image columns.
- Commonly, such images are then ***accessed in MATLAB by*** $I(M, N, channel)$ coordinates within the 3D array.

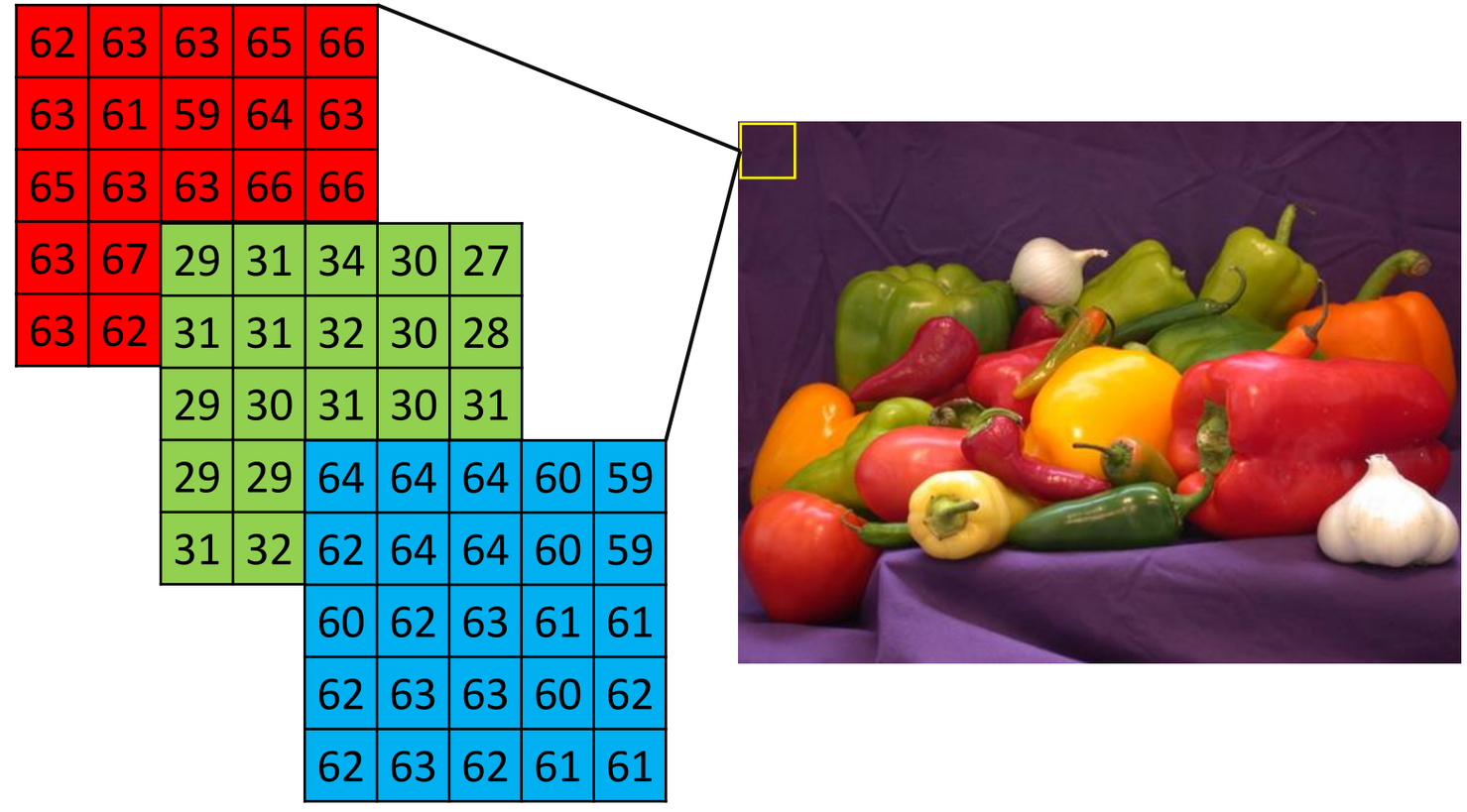


Image Data Types – Indexed Color Images

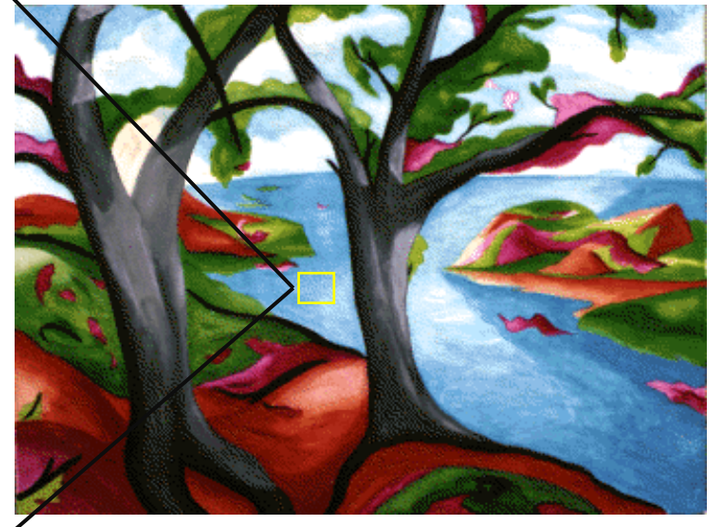
- **Indexed color images** constitute a very special class of color image.
- The difference between an ***indexed image and a truecolor image is the number of different colors*** (fewer for an indexed image) that can be used in a particular image.
- **In an indexed image**, ***the pixel values are only indices (pointers)*** with a maximum of 8 bits ***to a color palette (or color map).***
- **The color map** ***is simply a list of colors*** C of fixed maximum size (usually $C = 256$ colors) ***that are used in the image***.
- **The color map** is an $C \times 3$ ***array of class double containing values in the range*** $[0.0, 1.0]$. Each row of the color map specifies the red, green, and blue components of a single color.

Image Data Types – Indexed Color Images

- ***The pixel values of an indexed image must be integers*** in the range [0 , C]. The value 0 points to the first row in color map, the value 1 points to the second row, and so on.

94	0.419607843	0.678431373	0.870588235
95	0.580392157	0.709803922	0.352941176
96	0.482352941	0.678431373	0.776470588
97	0.647058824	0.678431373	0.450980392
98	0.709803922	0.580392157	0.807843137
99	0.580392157	0.709803922	0.611764706
100	0.937254902	0.517647059	0.741176471
101	0.580392157	0.709803922	0.741176471
102	0.937254902	0.549019608	0.647058824
103	0.937254902	0.580392157	0.517647059
104	0.776470588	0.741176471	0.290196078
105	0	1	1
106	0.549019608	0.741176471	0.905882353

<82> R:0.35 G:0.65 B:0.81	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87
<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87
<93> R:0.42 G:0.68 B:0.87	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87	<105> R:0.55 G:0.74 B:0.91	<93> R:0.42 G:0.68 B:0.87



An indexed color image and the indices in a 6 × 6 detailed region. Each pixel shows the index and the values of R, G, and B at the color map entry that the index points to

Image Compression

- Raw image representations usually require a large amount of storage space and proportionally long transmission times in the case of file uploads/downloads.
- ***Most image file formats employ some type of compression***. Thus, the main consideration in choosing an image storage format is compression.
- **Compressing an image** can mean ***it takes up less disk storage and can be transferred over a network in less time***.
- **Compression methods** can be classified into ***lossy compression and lossless compression*** techniques.

Image Compression – Lossy Compression

- **Lossy compression** operates by ***removing redundant information from the image***. Essentially, if such information is visually redundant then its transmission is unnecessary for appreciation of the image.
- **The information** that can be removed ***may be in terms of fine image detail*** or ***through a reduction in the number of colors/grey levels*** in a way that is not detectable by the human eye.
- Some of the image formats store the data in such a compressed form such as JPEG format.
- **Lossy compression** ***reduces the storage volume*** (sometimes dramatically) ***at the expense of some loss of detail*** in the original image.

Image Compression – Lossy Compression

- That is, **lossy compression** ***can significantly reduce the amount of image information*** that needs to be stored, but this can lead to ***a significant reduction in image quality***.
- This figure shows an example for the image that is compressed ***using varying levels of lossy compression***.

Original Image (8-bit RGB)
= 1024 x 768 x 3
= 2304Kb ~= 2.3Mb

JPEG (Quality : 0) = 16k

JPEG (Quality : 20) = 40k

JPEG (Quality : 75) = 168k

Image Compression – Lossless Compression

- **Lossless compression technique** is used in common image formats such as PNG format.
- In contrast to lossy compression, ***it can significantly reduce the amount of image information*** that needs to be stored, ***meanwhile***, it allows the original image to be ***reconstructed perfectly from the reduced data without any loss of image information***.
- This figure shows an example for the image that is compressed ***using lossless compression technique***.

Original Image (8-bit RGB)
= 1024 x 768 x 3
= 2304Kb ~= 2.3Mb

PNG (max. compression) = 1.4Mb

Image Compression

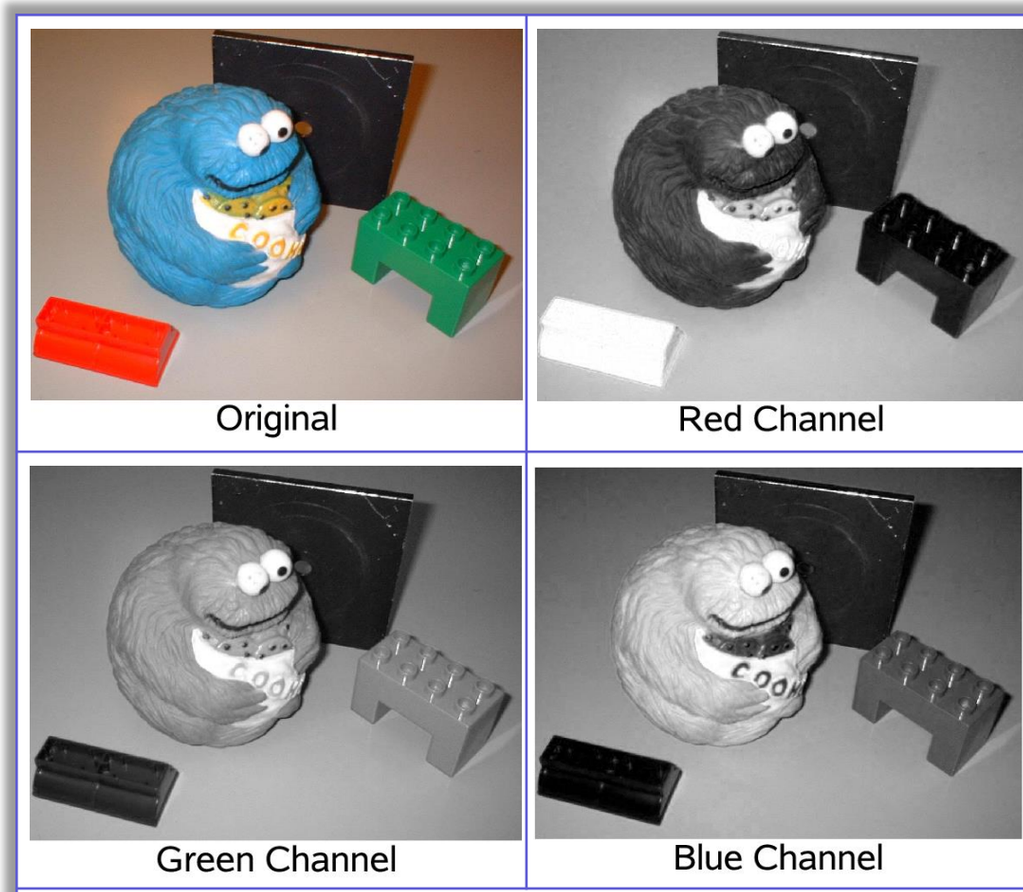
- As a general guideline, **lossy compression** should be used for general-purpose photographic images or ***images in which loss of detail may be acceptable***.
- **Lossless compression** should be preferred when dealing with ***images in which loss of detail may be unacceptable*** such as space images and medical images, faxes, etc.
- In terms of practical image processing in **MATLAB**, it should be noted that an image written to file from MATLAB in ***a lossy compression format*** (e.g. JPEG) will not be stored as the exact MATLAB image representation it started as. ***Image pixel values will be altered*** in the image output process as a result of the lossy compression. ***This is not the case if a lossless compression technique is employed.***

Color Spaces

- **Color space** means the use of a ***specific color model that turns colors into numbers*** (numerical values) that can be manipulated and processed.
- **Each color model** is a method of ***creating many colors from a group of primary colors***.
- Each model has a range of colors that it can produce. This range is the **color space**.
- **The representation of colors** in an image is achieved using ***a combination of number of color channels*** that are combined to form the color used in the image.
- The representation that is used to store the colors, ***specifying the number and nature of the color channels***, is generally known as **the color space**.

RGB Color Space

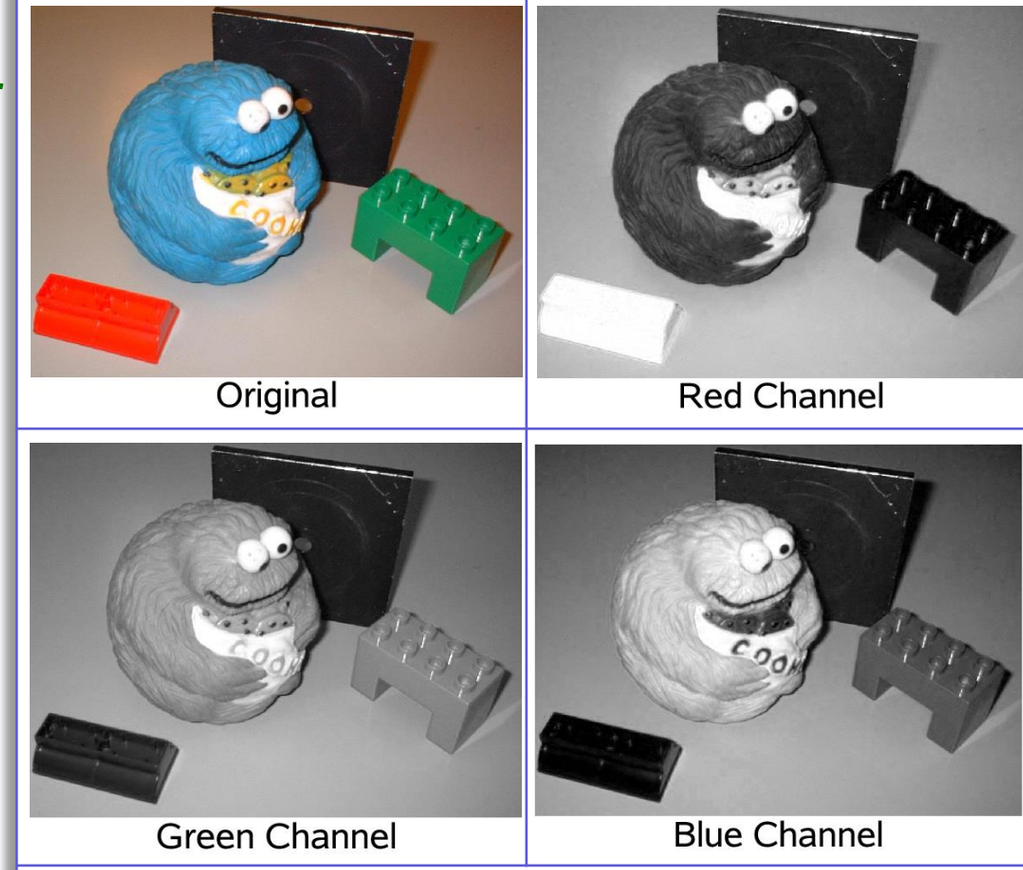
- RGB (or truecolor) images are 3D arrays that may be considered as three distinct 2D planes, one corresponding to each of the three R, G, and B color channels. RGB is ***the most common color space used for digital image representation*** as it conveniently corresponds to the three primary colors which are mixed for display on a monitor or similar device.
- ***It can easily separate and view*** the red, green and blue ***components of a truecolor image***, as shown in this figure.



Color image and its red, green, and blue components

RGB Color Space

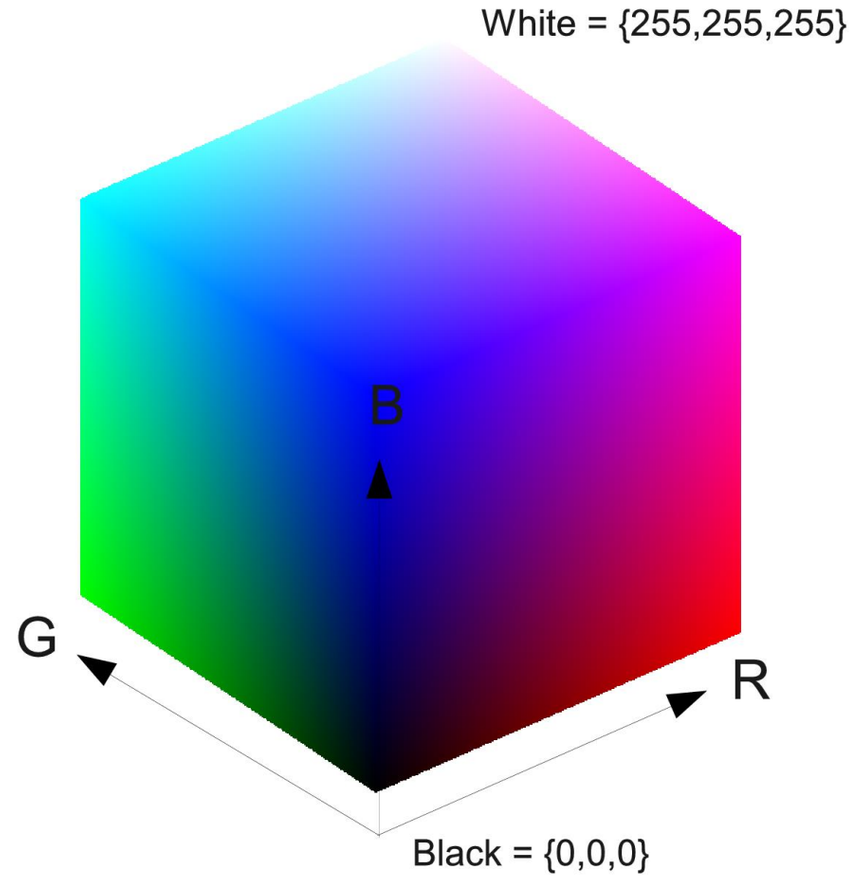
- It is important to note that the colors typically present in a real image are nearly always ***a blend of color components from all three channels***.
- A common **misconception** is that, for example, ***items that are perceived as blue will only appear in the blue channel*** and so forth. Whilst items perceived as blue will certainly appear ***brightest in the blue channel*** (i.e. they will contain more blue light than the other colors) they will also have milder components of red and green.



Color image and its red, green, and blue components

RGB Color Space

- If we consider all the colors that can be represented within the RGB representation, then we appreciate that the RGB color space is essentially a 3D color space (cube) with axes R, G, and B. Each axis has the range 0–255 for the common 1 byte per color channel (24-bit image representation).
- The color black occupies the origin of the cube (position (0,0,0)), corresponding to the absence of all three colours; white occupies the opposite corner (position (255,255,255)), indicating the maximum amount of all three colors. All other colors in the spectrum lie within this cube.



RGB to Greyscale Image Conversion

- An RGB color space can be converted to a greyscale image using a simple transform.
- ***Greyscale conversion is the initial step in many image analysis algorithms***, as it essentially simplifies (i.e. reduces) the amount of information in the image.
- Although a greyscale image contains less information than a color image, the majority of important feature related information is maintained such as edges, regions, blobs, junctions and so on. Feature detection and processing algorithms then typically operate on the converted greyscale version of the image.

RGB to Greyscale Image Conversion

- An RGB color image, I_{color}, is converted to greyscale image, $I_{greyscale}$, using the following transformation:

$$I_{greyscale}\,(m,n) = \alpha I_{color}\,(m,n,r) + \beta I_{color}(m,n,g) + \gamma I_{color}(m,n,b)$$

where (m,n) indexes an individual pixel within the greyscale image and (m,n,c) the individual channel at pixel location (m,n) in the color image for channel c in the red r, green g, and blue b image channels.

- **The greyscale image** is ***a weighted sum of the red, green, and blue color channels***. The weighting coefficients (α,β,γ) are set in proportion to the perceptual response of the human eye to each of the red, green and blue color channels.

RGB to Greyscale Image Conversion

- A standardized weighting ensures uniformity (NTSC television standard, $\alpha = 0.2989$, $\beta = 0.5870$, and $\gamma = 0.1140$).
- The human eye is naturally ***more sensitive to red and green light***. Thus, these colors are given higher weightings to ensure that the relative intensity balance in the resulting greyscale image is similar to that of the RGB color image.
- RGB to greyscale ***conversion is a noninvertible image transform***: the true color information that is lost in the conversion cannot be readily recovered.

Displaying Image Information – MATLAB

- The function **imfinfo** in **MATLAB** can be used to ***display information about image files*** including their type, format, size and bit depth, etc (without opening them and storing their contents in the workspace).
- The resulting structure (stored in variable ans) will contain the following information:

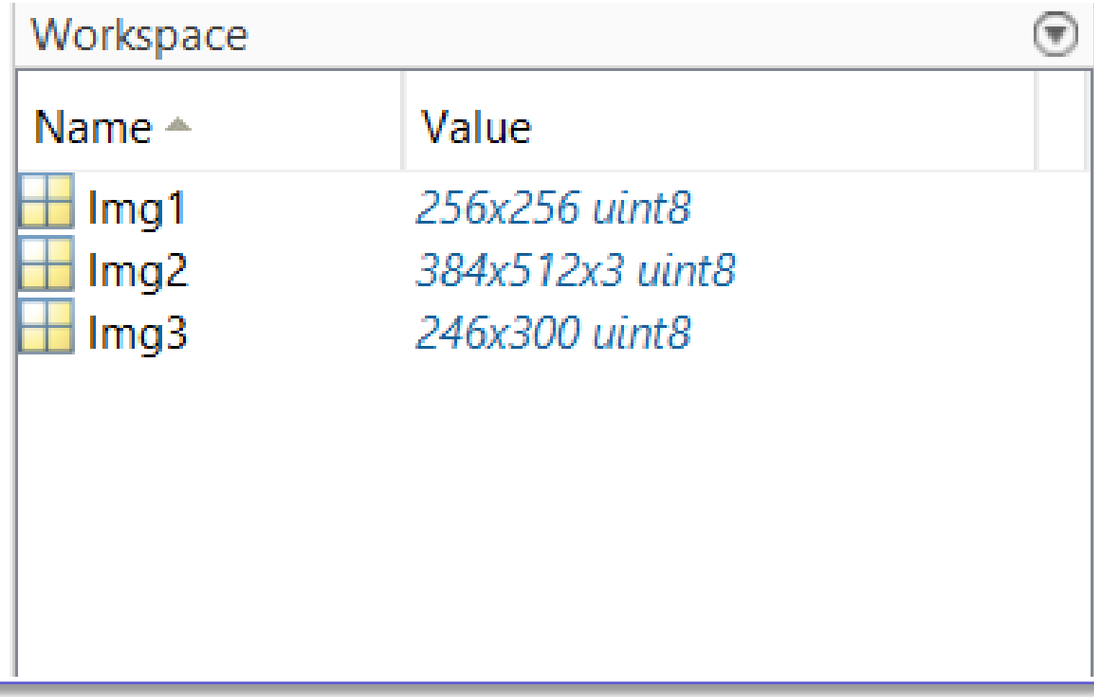
```
imfinfo('Peppers.png');
```

```
          Filename: 'C:\Program Files\MATLAB\…\peppers.png'
      FileModDate: '16-Dec-2002 03:10:58'
         FileSize: 287677
           Format: 'png'
            Width: 512
           Height: 384
         BitDepth: 24
        ColorType: 'truecolor'
  FormatSignature: [1×8 double]
    InterlaceType: 'none'
     Transparency: 'none'
     ImageModTime: '16 Jul 2002 16:46:41 +0000'
      Description: 'Zesty peppers'
        Copyright: 'Copyright The MathWorks, Inc.'
```

Reading Images – MATLAB

- **MATLAB** has a built-in function called **imread** that can be used to ***open and read the contents of image files*** in most common formats (e.g., TIFF, JPEG, BMP, GIF, and PNG).
- The imread function creates/exports the appropriate 2D/3D image arrays within the MATLAB environment via storing them into a variable in the MATLAB workspace.

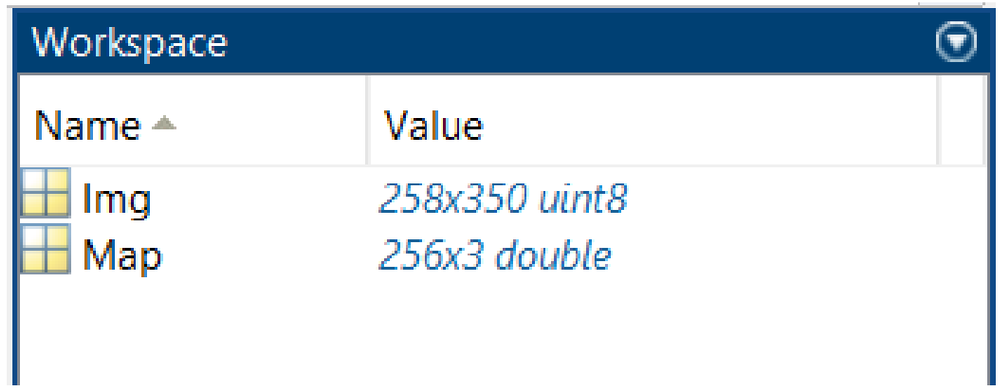
```
Img1 = imread('cameraman.tif');
Img2 = imread('Peppers.png');
Img3 = imread('coins.png');
```



Reading Images – MATLAB

- In **MATLAB**, to read in an indexed image, we must specify variables for both the image and its color map.

```
[Img,Map] = imread('trees.tif');
```



Writing Images – MATLAB

- **MATLAB** has a built-in function called **imwrite** that can be used to ***write the contents of an image*** in one of the most common graphic file formats.
- If the output file format uses ***lossy compression*** (e.g., JPEG), imwrite allows the specification of a ***quality parameter***, used as a trade-off between the resulting image's subjective quality and the file size.

```
Img1 = imread('Peppers.png');
imwrite(Img1, 'PepersQuality_75.jpg');
imwrite(Img1, 'PepersQuality_05.jpg', 'quality', 5);
imwrite(Img1, 'PepersQuality_95.jpg', 'quality', 95);
```

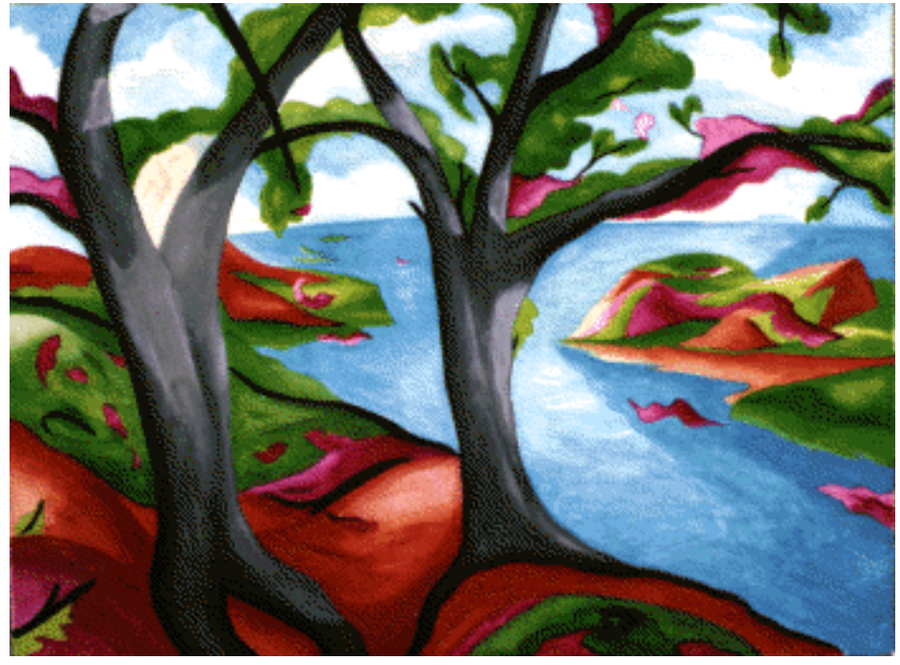
Writing Images – MATLAB

- In this example, we will read an image from a PNG file and save it to a JPG file using three different quality parameters: 75 (default), 5 (poor quality, small size), and 95 (better quality, larger size).

			
Original image	Compressed image (JPG, q = 75, file size = 24 kB)	Compressed image (JPG, q = 5, file size = 8 kB)	Compressed image (JPG, q = 95, file size = 60 kB)

Writing Images – MATLAB

- To write the contents of an ***indexed image***, we must pass both ***the image and its color map*** to imwrite function.

`imwrite(Img, 'TreesOriginal.tif');`	`imwrite(Img,Map, 'TreesOriginal.tif');`
	

Basic Display of Images – MATLAB

- **MATLAB** has several functions for displaying images:.
  - **image**: ***displays an image using the current color map***. The color map array is an M × 3 matrix of class double, where each element is a floating-point value in the range [0, 1]. Each row in the color map represents the R (red), G (green), and B (blue) values for that particular row.
  - **imagesc**: accepts input arrays of any MATLAB storage type (uint8, uint16 or double) and any numerical range. It then ***scales image data to the full range of the current/default color map and displays the image***.

Basic Display of Images – MATLAB

- **MATLAB** has several functions for displaying images:.
  - **imshow**: displays an image and contains a number of optimizations and optional parameters for property settings associated with image display.
  - **imtool:** displays an image and contains a number of associated tools that can be used to explore the image contents.

Basic Display of Images – MATLAB

- Displaying intensity images using different **imshow** options:

  (a) Opens a new figure and displays the image in its original state.

  (b) Opens a new figure and displays a scaled version of the same image.

```
Img = imread('pout.tif');
figure,imshow(Img)
```

(a)

```
Img = imread('pout.tif');
figure,imshow(Img,[])
```

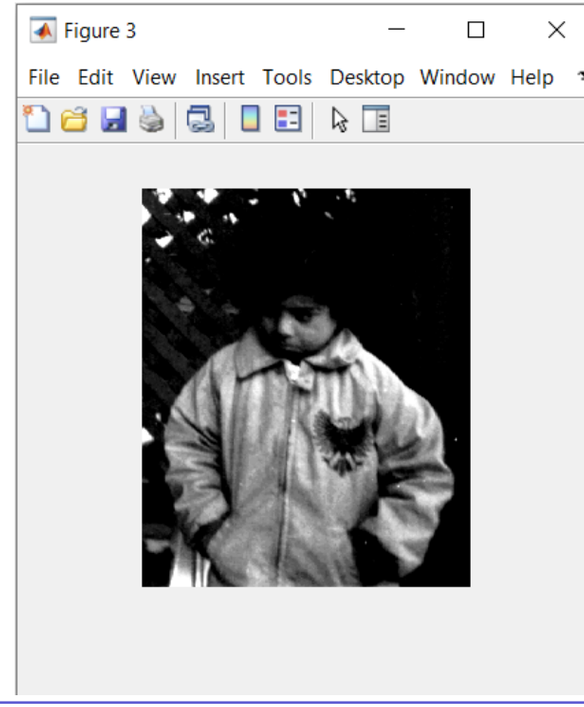
(b)

Basic Display of Images – MATLAB

- Displaying intensity images using different **imshow** options:

  (c) Opens a new figure and specifies a range of gray levels, such that all values lower than 100 will be displayed as black and all values greater than 160 will be displayed as white.

```
Img = imread('pout.tif');
figure,imshow(Img,[100 160])
```

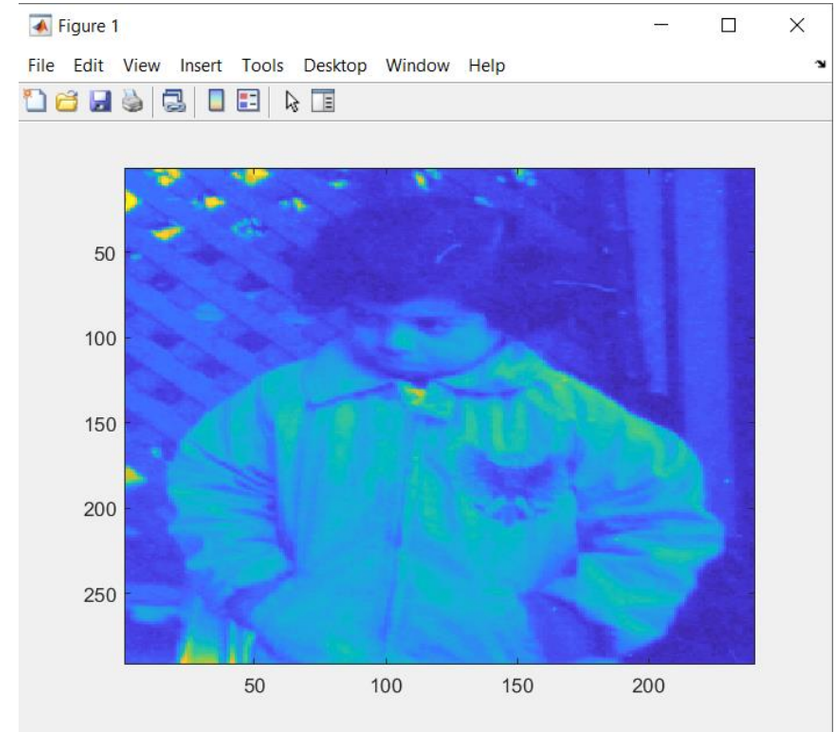


(c)

Basic Display of Images – MATLAB

- Displaying intensity (greyscale) image using **imagesc** function:

```
Img = imread('pout.tif');
figure,imagesc(Img)
```

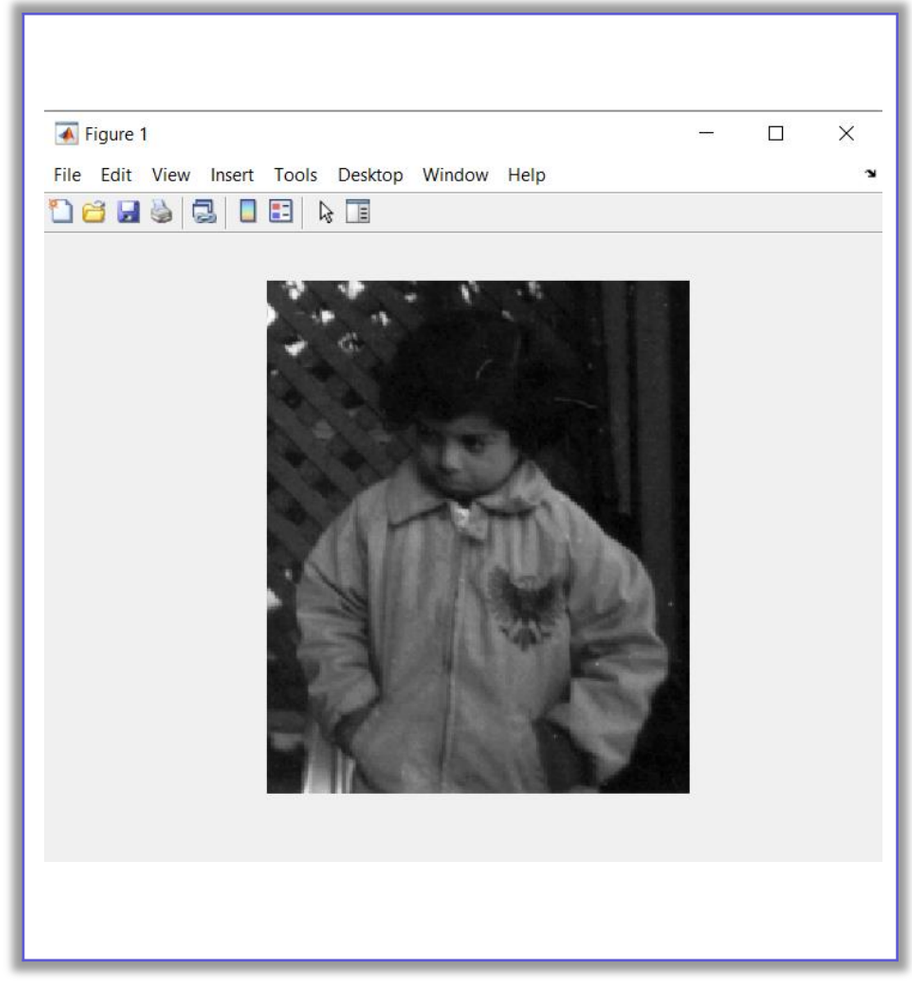


- Note additional steps are required when using **imagesc** function to display conventional images.
- We can control the current/default color map using the **colormap** function in **MATLAB**.

Basic Display of Images – MATLAB

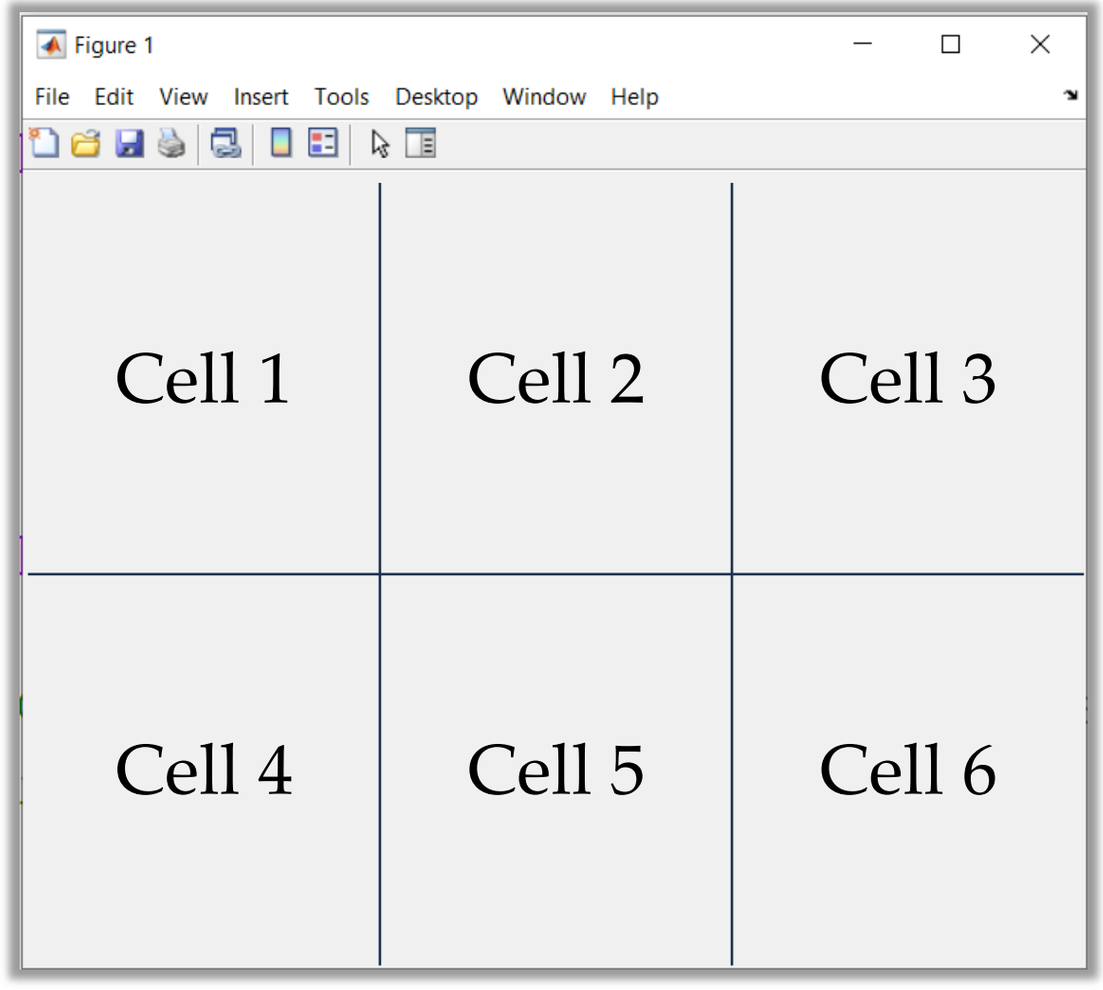
- Displaying intensity (greyscale) image using **imagesc** function:

```
Img = imread('pout.tif');
figure,imagesc(Img)
axis image; % Correct aspect ratio of displayed image
axis off; % Turn off the axis labelling
colormap(gray) % Display intensity image in grey scale
```



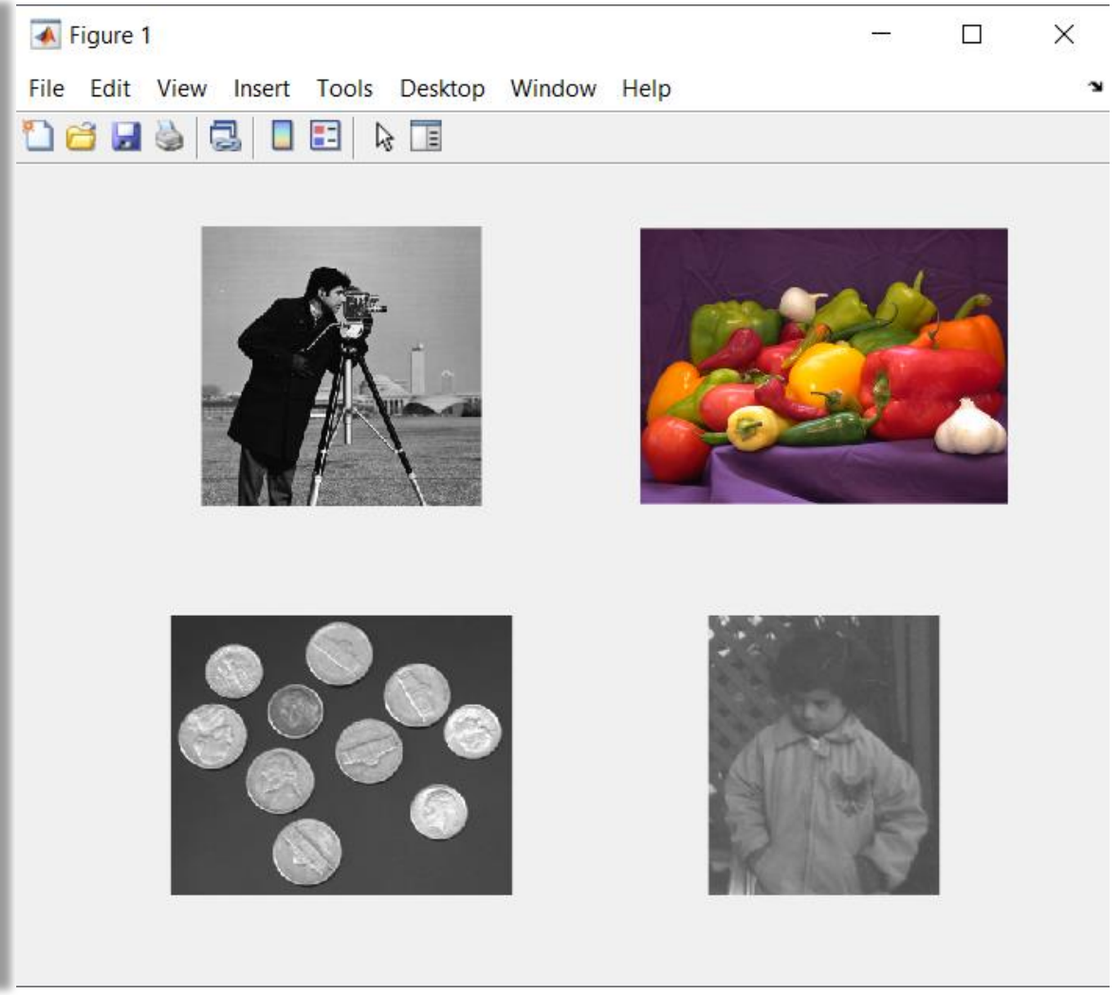
Basic Display of Images – MATLAB

- **MATLAB** has a built-in function called **subplot** that can be used to display multiple images within one figure.
- When using this function, the first two parameters specify the number of rows and columns to divide the figure. The third parameter specifies which subdivision to use.
- In the case of **subplot(2, 3, 3)**, we are telling **MATLAB** to divide the figure into two rows and three columns and set the third cell as active.



Basic Display of Images – MATLAB

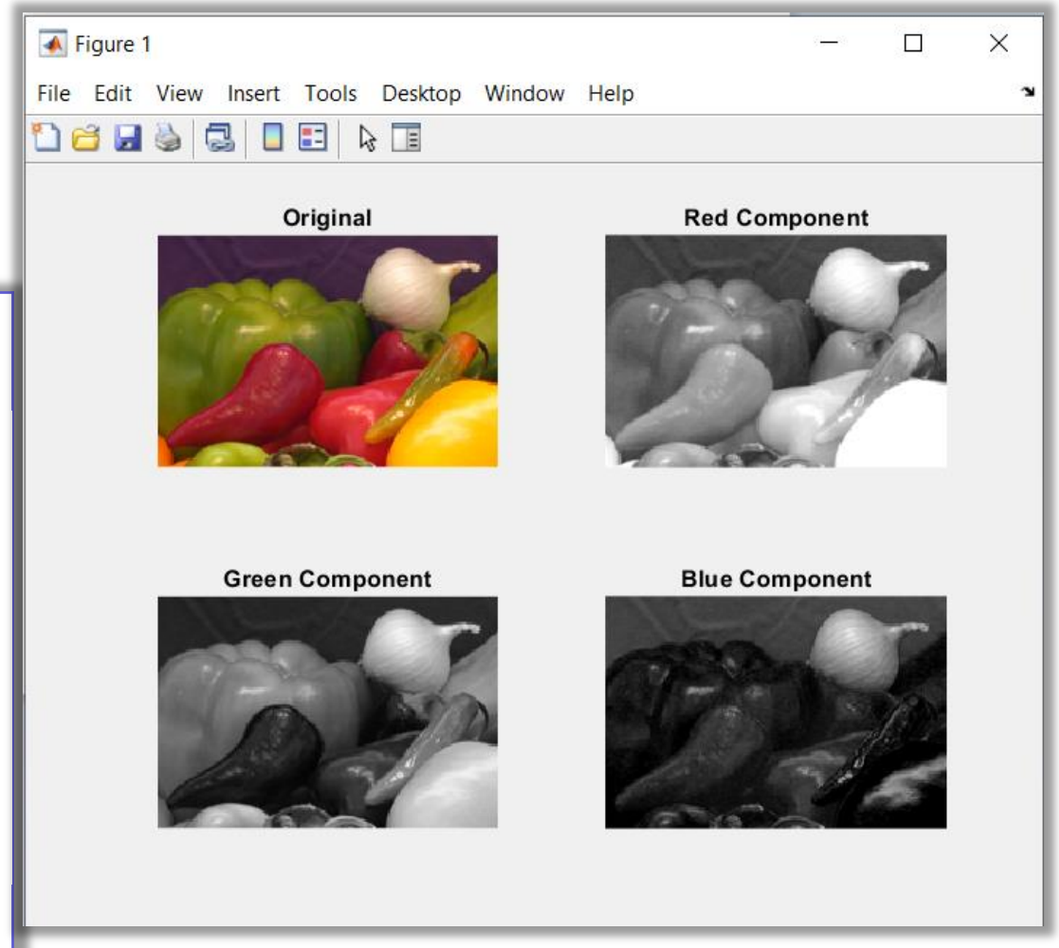
```
Img1 = imread('cameraman.tif');
Img2 = imread('Peppers.png');
Img3 = imread('Coins.png');
Img4 = imread('Pout.tif');
figure
subplot(2,2,1),imshow(Img1)
subplot(2,2,2),imshow(Img2)
subplot(2,2,3),imshow(Img3)
subplot(2,2,4),imshow(Img4)
```



Basic Display of Images – MATLAB

- We can access individual channels of an RGB image and extract them as separate images in their own right.

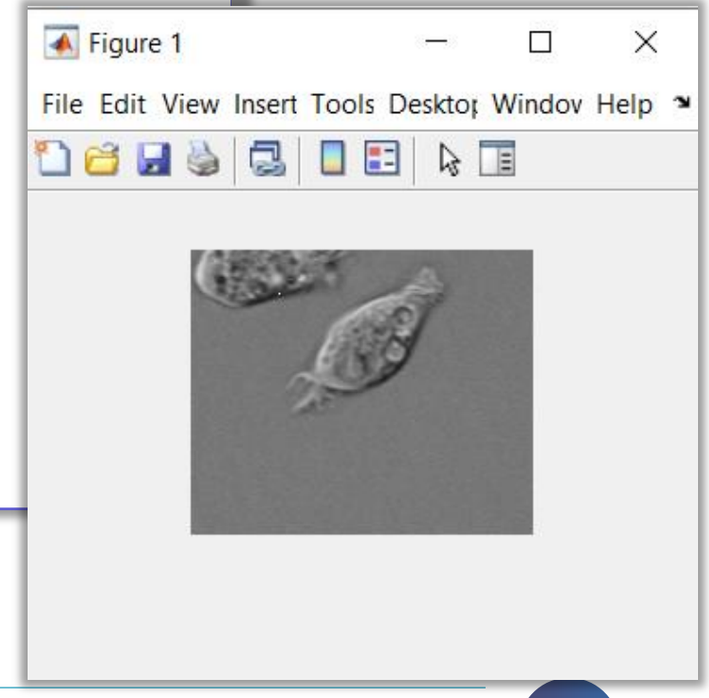
```
Img = imread('Onion.png');%Read in 8 bit RGB color image
ImgRed = Img(:,:,1);%Extract red channel (first channel)
ImgGreen = Img(:,:,2);%Extract green channel (second channel)
ImgBlue = Img(:,:,3);%Extract blue channel (third channel)
figure
subplot(2,2,1),imshow(Img);title('Original');
subplot(2,2,2),imshow(ImgRed);title('Red Component');
subplot(2,2,3),imshow(ImgGreen);title('Green Component');
subplot(2,2,4),imshow(ImgBlue);title('Blue Component');
```



Accessing Pixel Values – MATLAB

- We can ***access individual pixel values*** within the image and change their value.
- **Example 1:**

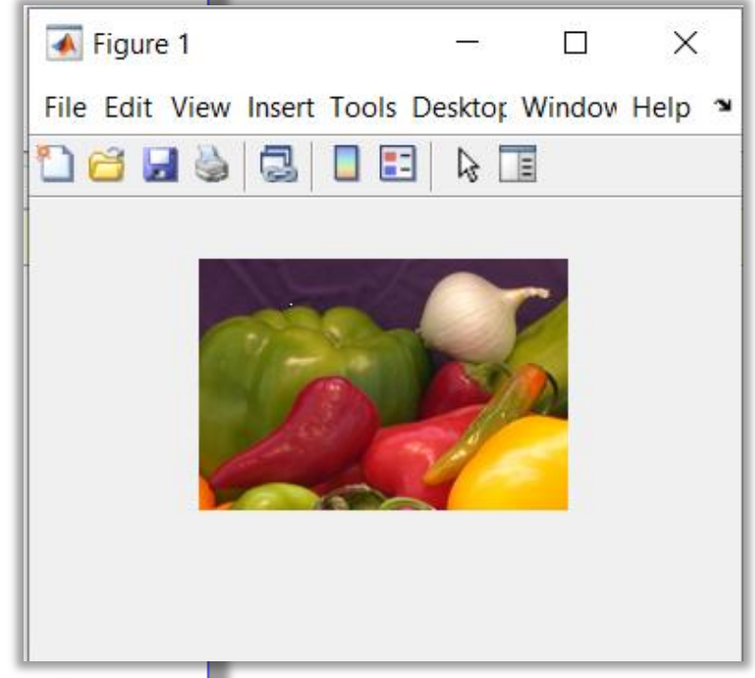
```
Img1 = imread('cell.tif'); %Read in 8 bit intensity image of cell
Img1(25,50) %Print pixel value at location (25,50)
ans =
   uint8
    46
Img1(25,50) = 255; %Set pixel value at (25,50) to white
imshow(Img1); %View resulting changes in image
```



Accessing Pixel Values – MATLAB

- We can ***access individual pixel values*** within the image and change their value.
- **Example 2:**

```
Img2 = imread('onion.png'); %Read in 8 bit color image
Img2(25,50,:) %Print RGB pixel value at location (25,50)
    1×1×3 uint8 array
    ans(:,:,1) = 46
    ans(:,:,2) = 29
    ans(:,:,3) = 50
Img2(25,50, 1) %Print only the red value at (25,50)
    ans =
      uint8
        46
Img2(25,50,:) = [255, 255, 255]; %Set pixel value to RGB white
imshow(Img2);
```



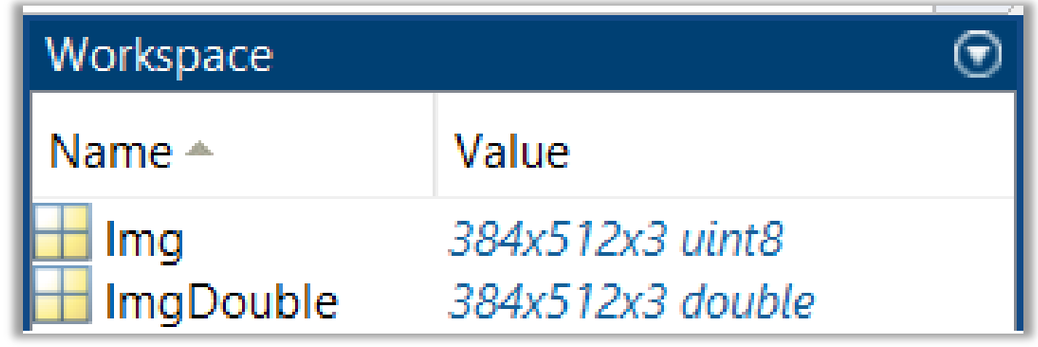
Converting Image Types – MATLAB

- The most common data classes for images in **Matlab** are as follows:
  - **uint8:** 1 byte per pixel, in the [0, 255] range
  - **uint16:** 2 byte per pixel, in the [0, 65535] range
  - **double:** 8 bytes per pixel, usually in the [0.0, 1.0] range
  - **logical:** 1 byte per pixel, representing its value as true (1 or white) or false (0 or black)

Converting Image Types – MATLAB

- **Example:**

```
Img = imread('Peppers.png'); %Img is an RGB image of type uint8
ImgDouble = im2double(Img); %Convert Img to double type
```



- We can verify that the new image consists of pixel values in the range [0, 1].

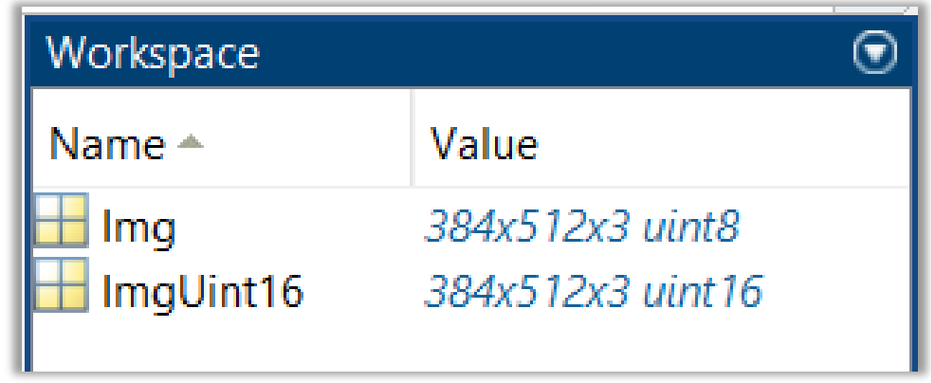
```
min(ImgDouble(:))
ans = 0
```

```
max(ImgDouble(:))
ans = 1
```

Converting Image Types – MATLAB

- **Example:**

```
Img = imread('Peppers.png'); %Img is an RGB image of type uint8
ImgUint16 = im2uint16(Img); %Convert Img to uint16 type
```



- We can verify that the new image consists of pixel values in the range [0, 65535].

```
min(ImgUint16(:))
ans =
  uint16
    0
```

```
max(ImgUint16(:))
ans =
  uint16
  65535
```

Converting Image Types – MATLAB

- **MATLAB** also has a built-in function called **im2bw** that can be used to convert images to binary equivalents.

ImgB = **im2bw**(Img , **Level**);

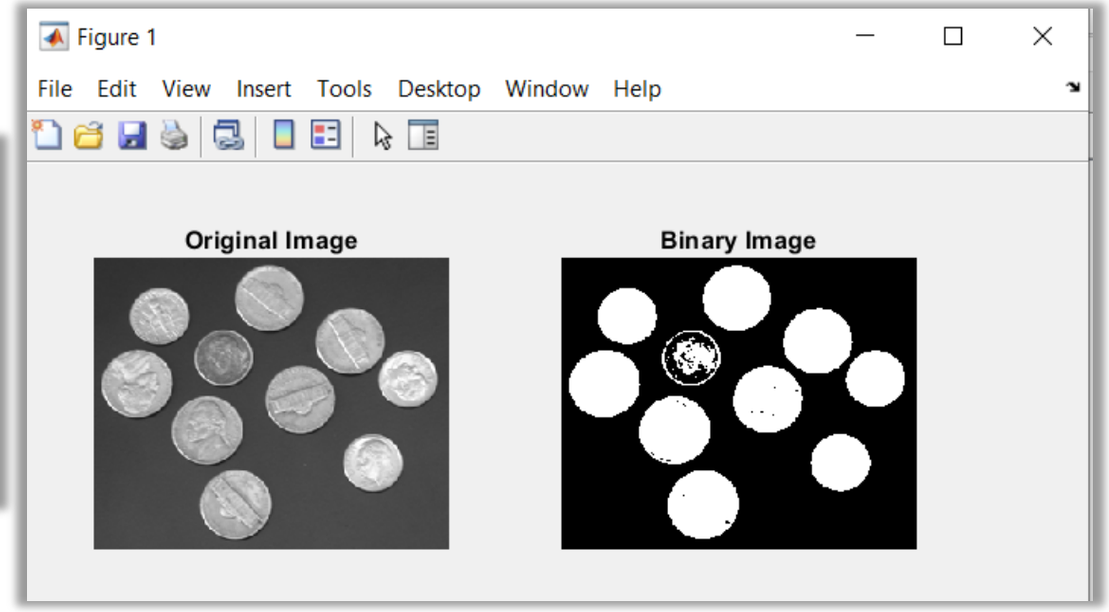
ImgB = **im2bw**(IndexedImg , Map , **Level**);

- **im2bw** function converts the grayscale/color image or indexed image to binary image, by replacing all pixels in the input image with luminance greater than **Level** with the value 1 (white) and replacing all other pixels with the value 0 (black). **Level** is a luminance threshold which is a ***nonnegative number between 0 and 1*** and the default value is 0.5.

Converting Image Types – MATLAB

- **Example:**

```
Img = imread('Coins.png');
ImgB = im2bw(Img);
figure
subplot(1,2,1),imshow(Img);title('Original Image');
subplot(1,2,2),imshow(ImgB);title('Binary Image');
```

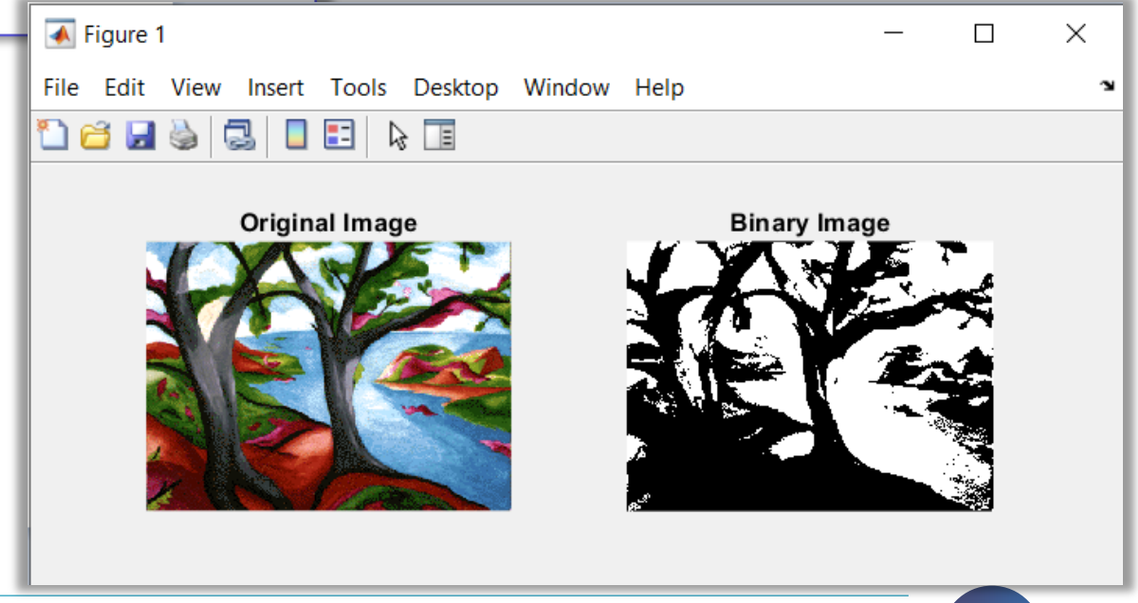


- Any value greater than 0.5 becomes 1 (true), otherwise it becomes 0 (false). Note that since im2bw expects a threshold luminance level as a parameter, and requires it to be a nonnegative number between 0 and 1, it implicitly assumes that the input variable (Img) is a ***normalized grayscale image of class double***.

Converting Image Types – MATLAB

- **Example:**

```
[Img,Map] = imread('trees.tif');
ImgB = im2bw(Img,Map,0.4);
figure
subplot(1,2,1),imshow(Img,Map);title('Original Image');
subplot(1,2,2),imshow(ImgB);title('Binary Image');
```



- Any value greater than 0.4 becomes 1 (true), otherwise it becomes 0 (false).

Converting Image Types – MATLAB

- Some operations may require you to convert an image from one type to another.
- **MATLAB** also includes functions to convert between RGB (truecolor), indexed image, and grayscale image.

Name	Converts	Into
ind2gray	An indexed image	Its grayscale equivalent
gray2ind	A grayscale image	An indexed representation
rgb2gray	An RGB (truecolor) image	Its grayscale equivalent
rgb2ind	An RGB (truecolor) image	An indexed representation
ind2rgb	An indexed color image	Its RGB (truecolor) equivalent

Converting Image Types – MATLAB

- We can also convert ***RGB image to a grayscale image*** as follows:

```
GrayImg = rgb2gray( RGBImg );
```

- We can convert ***RGB image to an indexed image*** as follows:

```
[IndexedImg , Map] = rgb2ind( RGBImg , Q );
```

where **Q** denotes the number of quantized colors used for minimum variance quantization, specified as a positive integer that is less than or equal to 65,536. The returned colormap **Map** has **Q** or fewer colors.

Converting Image Types – MATLAB

- **Example:**

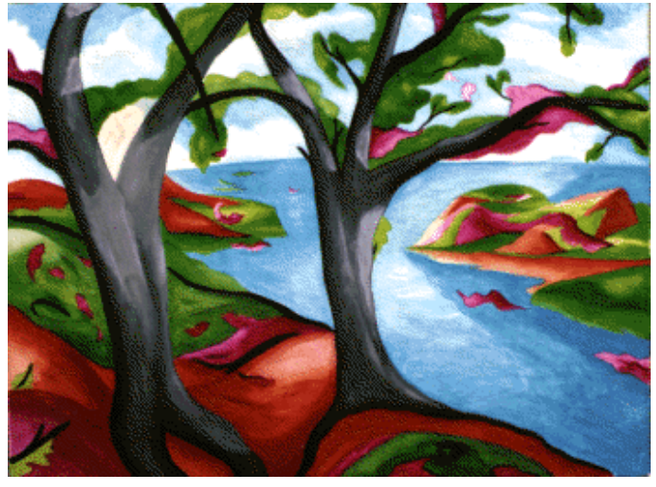
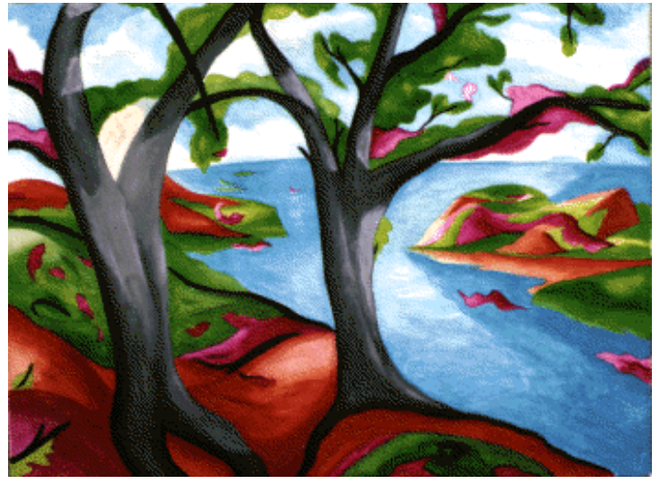
```
RGBImg = imread('Peppers.png'); %Read in an RGB image
GrayImg = rgb2gray(RGBImg); %Convert RGB image to grayscale image
IndexedImg = rgb2ind(RGBImg,50); %Convert RGB image to indexed image
```

RGB Image	Greyscale Image	Indexed Image with 50 quantized colors
		

Converting Image Types – MATLAB

- **Example:**

```
[IndexedImg,Map] = imread('trees.tif');%Read in an indexed image
RGBImg = ind2rgb(IndexedImg,Map);%Convert an indexed image to RGB image
GrayImg = ind2gray(IndexedImg,Map);%Convert an indexed image to intensity image
```

Indexed Image	RGB Image	Greyscale Image
		

Exploring Contents of Images – MATLAB

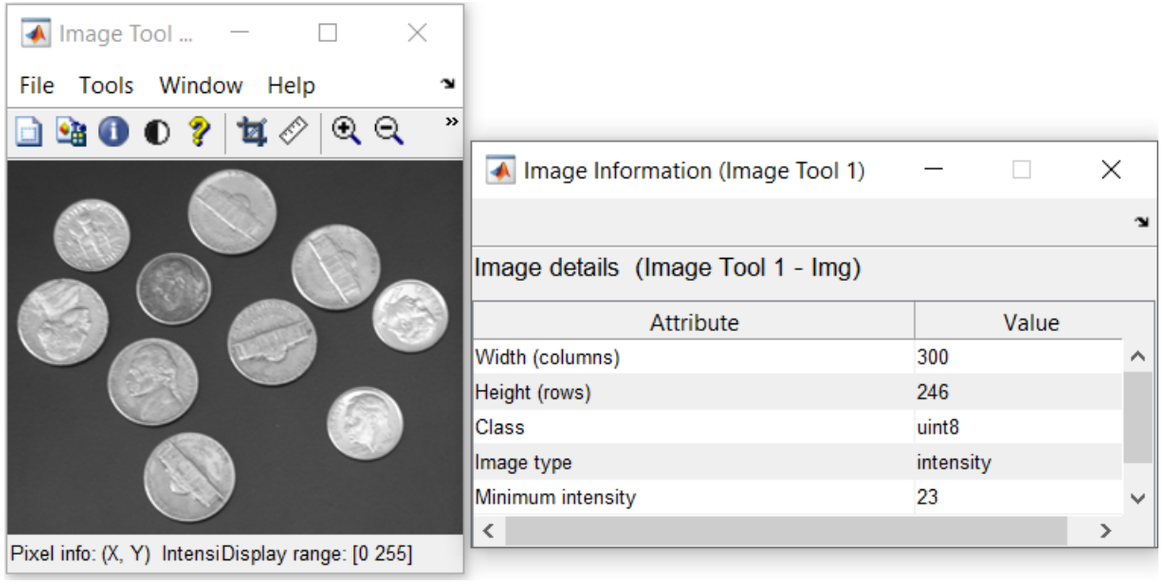
- MATLAB has a built-in function called **imtool** that can be used to ***inspect the contents of an image more closely***.
- This function provides all the image display capabilities of **imshow** as well as access to other tools for navigating and exploring images, such as the ***Pixel Region tool***, ***Image Information tool***, and ***Adjust Contrast tool***.
- These tools can also be directly accessed using the **imshow** function with their library functions **impixelinfo**, **imageinfo**, and **imcontrast**, respectively.

Exploring Contents of Images – MATLAB

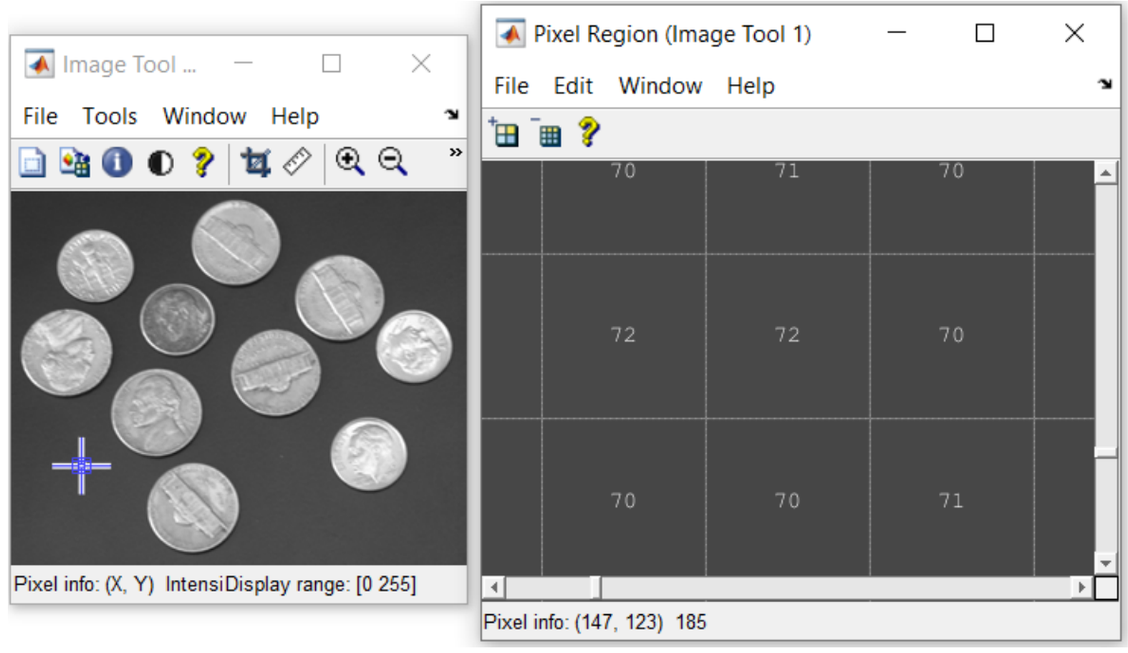
- **Example:**

```
Img = imread('Coins.png');
imtool(Img)
```

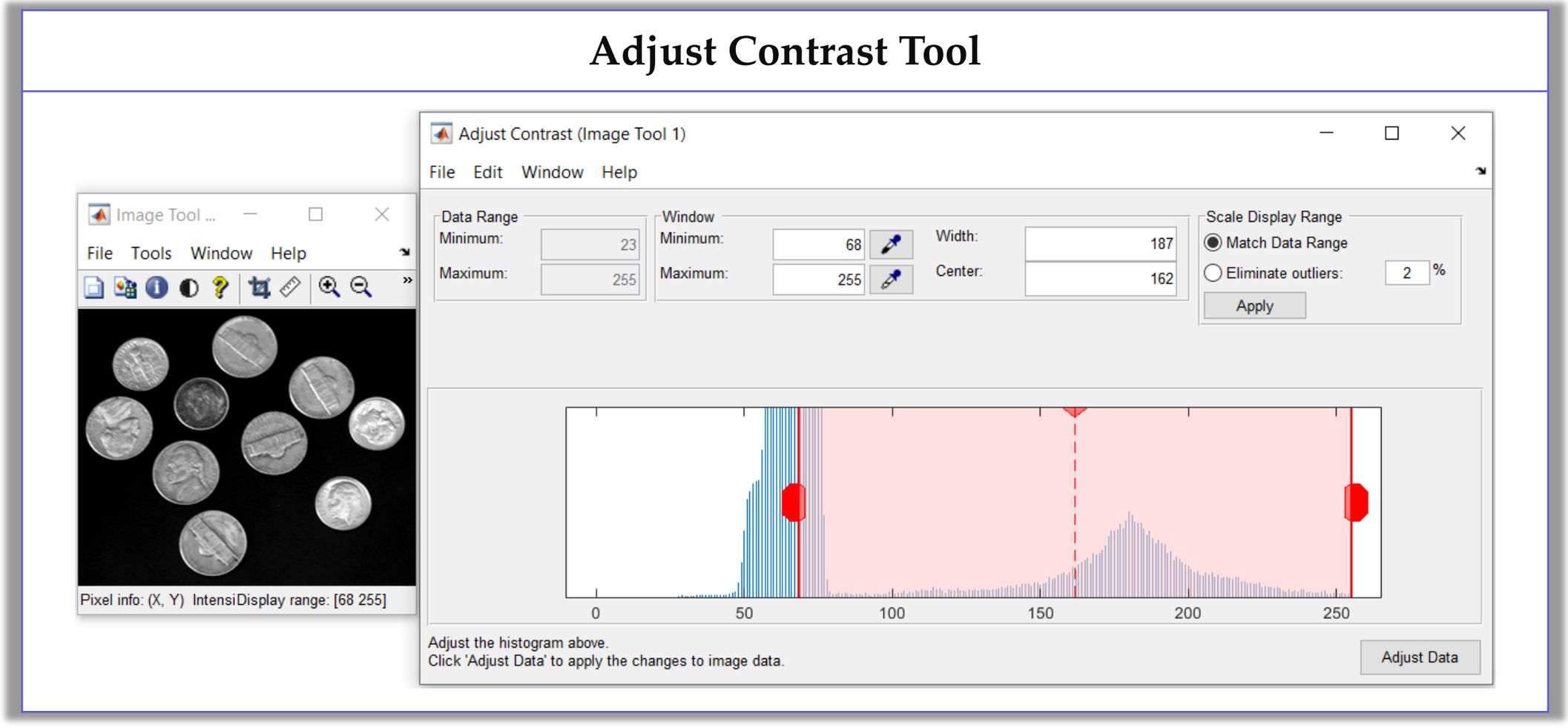
Image Information Tool



Pixel Region Tool



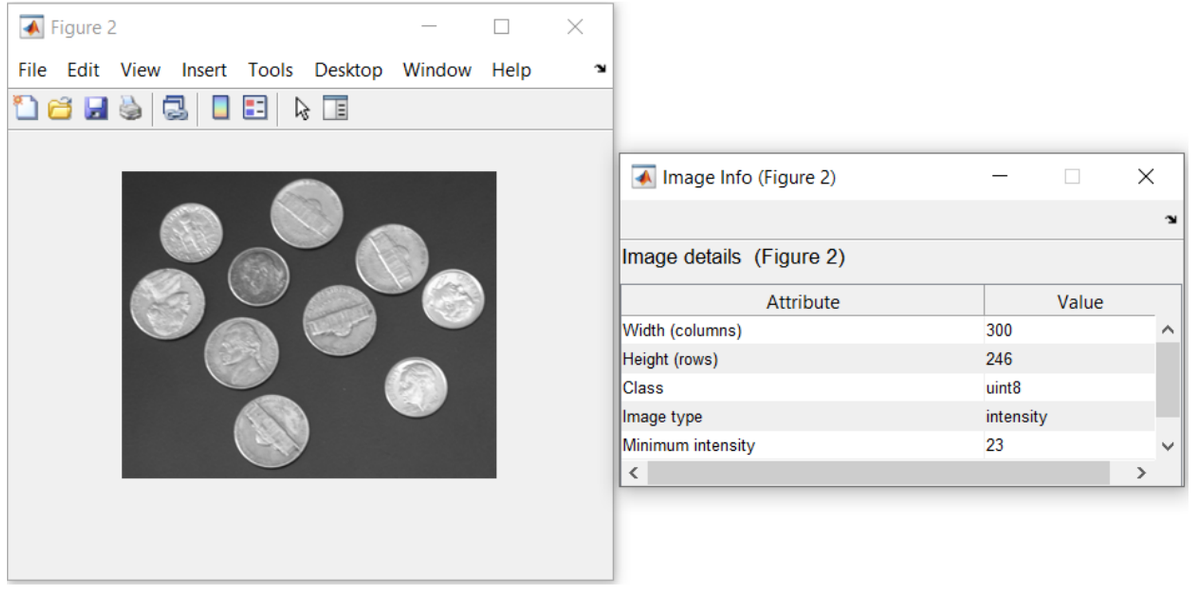
Exploring Contents of Images – MATLAB

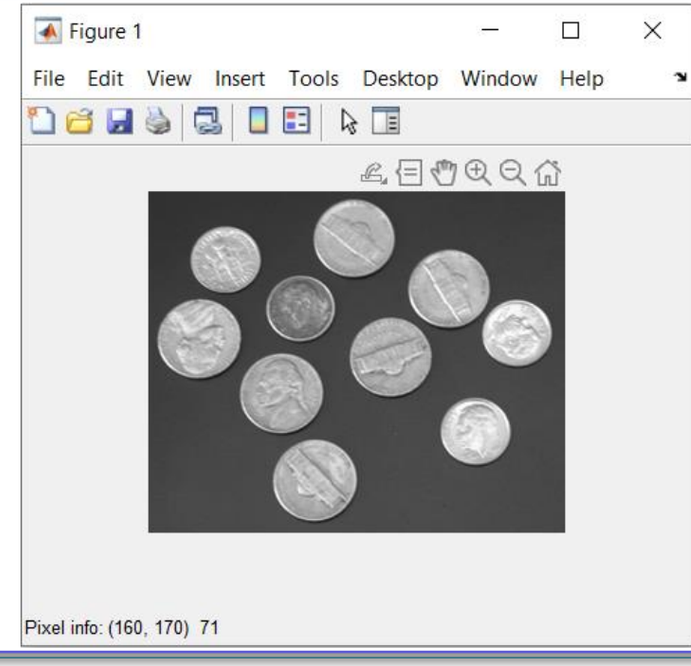


Exploring Contents of Images – MATLAB

- **Example:**

Image Information Tool	Pixel Region Tool
`Img = imread('Coins.png');` `figure, imshow(Img), imageinfo`	`Img = imread('Coins.png');` `figure, imshow(Img), impixelinfo`

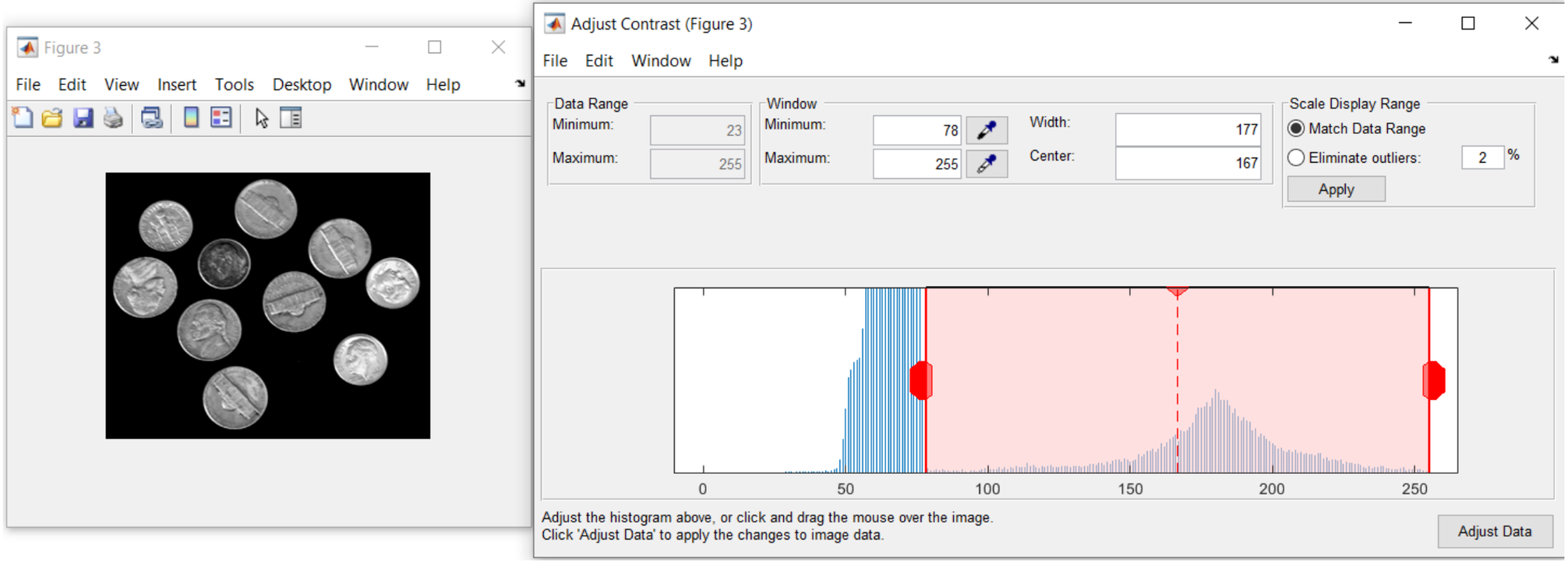




Exploring Contents of Images – MATLAB

Adjust Contrast Tool

```
Img = imread('Coins.png');
figure, imshow(Img), imcontrast
```




Thank
You